Highlights

Digital Twins for Manufacturing Systems: A Case Study Based On a Fischertechnik Factory

MBDO Dream Team,

-
-
-

Digital Twins for Manufacturing Systems: A Case Study Based On a Fischertechnik Factory⋆

MBDO Dream Team[a,*,1], [b,2]

[a], , , , ,
[b], , , , ,

ARTICLE INFO

Keywords:

ABSTRACT

Here goes the abstract

1. Introduction (Benoit)

TBC. @Benoit

https://docs.google.com/document/d/1ewH6IXOo0aI8pxRX4CZSeuyVqYTuUPgTan-SsxxQ/edit?usp=sharing

2. Fischertechnik Physical System Description

2.1. Physical Setup and Control devices

The demonstrator realizes a compact modular production island that combines storage, transport, processing, and quality inspection in a single reusable setup. This composition is sufficient to study digital-twin synchronization, control integration, and adaptive services without requiring a full-scale factory. The physical demonstrator consists of multiple fischertechnik modules arranged as a compact production line, as depicted in Figure 1. The setup includes the stations *HighBayWarehouse01*, *VacuumGripper01*, *VacuumGripper02*, *ConveyorBelt01*, *MultiProcessingStation01*, and *SortingLine01*. In contrast to using several separate product-specific lines, the demonstrator integrates different processing steps into one shared line in order to support flexibility, compactness, and efficient use of hardware.

duce
tails in
sML
gram.

Figure 2 presents a SysML structure of the overall system as a hierarchy of owned parts. The model is organized into two package-level composition contexts. The left-hand context contains the larger set of constituent blocks and represents the main structural backbone of the system. Most of these elements are modeled as leaf-level blocks with their own internal feature sets, while a dedicated aggregation block located in the lower central area of the package collects them through composition relations. The composition links make the ownership semantics explicit: the connected elements are not modeled as independent peers, but as constituent parts of the enclosing subsystem. This yields a clear separation between reusable element definitions at the upper level and their concrete integration into one subsystem-level structure at the lower level.

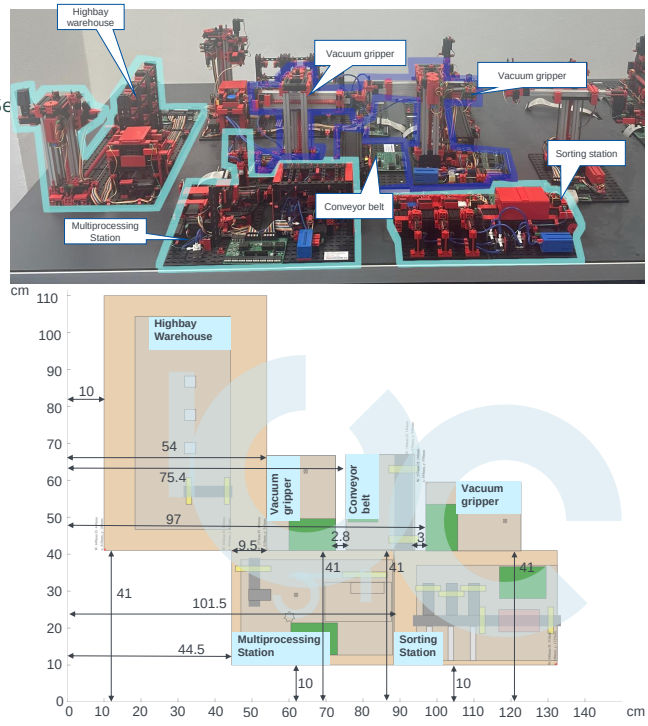


Figure 1: Fischertechnik demonstrator: (a) physical setup, (b) dimensioned top-view layout used for digital-twin synchronization.

The right-hand context follows the same principle on a smaller scale. Here, three higher-granularity blocks are arranged as the primary constituents of the package and are again merged into a dedicated subsystem block through composition. Compared with the left-hand context, this side of the model emphasizes coarser-grained structural units rather than a larger number of fine-grained elements. The repeated use of the same composition pattern in both contexts shows that the system is built from modular building blocks that can be grouped into intermediate subsystems before being integrated into the complete solution.

At the next abstraction level, the two subsystem blocks are connected to a single system-level block at the bottom of the diagram. This final composition step establishes the overall part–whole hierarchy of the model: elemental blocks

⋆

*Corresponding author
(M.D. Team); ( )
(M.D. Team); ( )
ORCID(s):
1

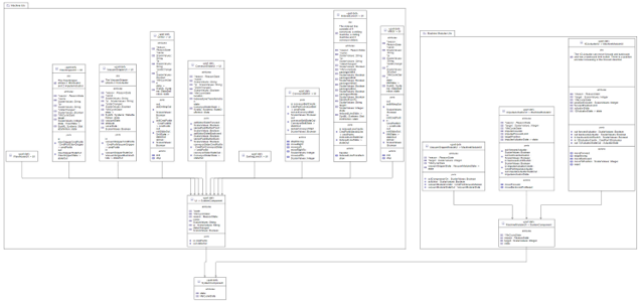


Figure 2: SysML model of the system architecture. The diagram organizes fine-grained and coarse-grained blocks into two subsystem-level composition contexts, which are subsequently integrated into one overall system block through explicit part–whole relations.

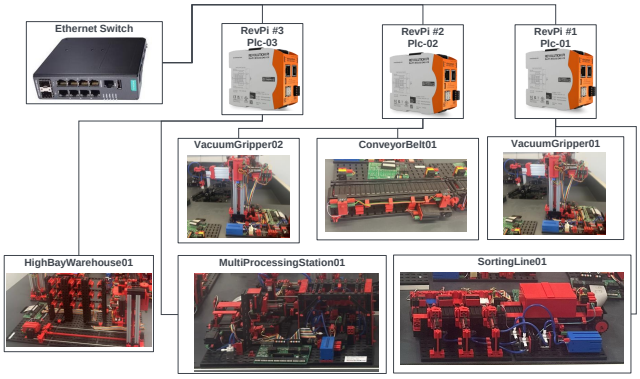


Figure 3: Cabling and controller assignment of the fischertechnik demonstrator, showing the RevPi-based PLC nodes (Plc-01 to Plc-03), Ethernet switch, and connections to the physical stations.

are first consolidated into subsystem structures, and these subsystem structures are then combined into the complete system. The diagram therefore documents not only the existence of individual structural elements, but also their ownership boundaries, integration path, and refinement logic across levels of abstraction. In a paper context, this figure can thus be used to argue for a modular system design in which traceable composition links relate local element definitions to subsystem formation and, ultimately, to the integrated overall architecture.

Conceptually, the line represents a simplified manufacturing process in which raw material carriers are transformed into intermediate and final products through a sequence of handling and processing steps. The exact product composition and process path may vary depending on concrete use case. In the demonstrator, this variability is represented in a simplified form so that the physical setup remains understandable while still reflecting the core characteristics of a modular production environment.

In the reference scenario used throughout this paper, a carrier is retrieved from *HighBayWarehouse01*, transferred by a vacuum gripper to *ConveyorBelt01*, processed in *MultiProcessingStation01*, returned to the conveyor, and finally inspected and routed by *SortingLine01*.

VacuumGripper01 primarily mediates between storage and transport, whereas *VacuumGripper02* mediates between transport and downstream processing/sorting steps. The *ConveyorBelt01* provides the main transport path between stations and serves as the shared transport backbone of the line. In addition, the conveyor acts as an intermediate handover point between handling and processing operations.

The *MultiProcessingStation01* represents the main processing unit in the line. In the demonstrator scenario, it is used to model preparation and transformation steps such as mixing, heating, and forming. After processing, the intermediate product is returned to *ConveyorBelt01* and transported to the next handling point. This enables a cyclic interaction between transport and processing, which is typical for modular production systems.

At the end of the line, the product is transferred to *SortingLine01*, which represents the quality inspection and sorting stage. Here, product properties are checked and the product is routed according to the inspection result. In the simplified demonstrator setting, this station captures the essential idea of quality-dependent branching in a production process, including the distinction between accepted and rejected parts.

The control of the physical setup is realized through RevPi-based PLC nodes that coordinate the stations as a production island. As shown in the cabling diagram (cf. Figure 3), the setup uses three controllers, labeled *Plc-01*, *Plc-02*, and *Plc-03*, each implemented on a RevPi and connected through an Ethernet switch. The RevPis are wired to the fischertechnik stations and provide the hardware interface to motors, sensors, and other station signals. In this role, they form the bridge between the physical demonstrator and the software stack: they acquire sensor and state information from the machines and execute control commands on the actuators.

2.2. Interfaces APIs of Physical Modules

At the control layer, stations expose low-level digital I/O signals. At the communication layer, these signals are made available via sockets or MQTT. At the digital-twin layer, higher-level commands and derived states are represented as machine-readable properties and methods. High-level actions such as *pickup(x,y,z)* are decomposed by the control layer into sequences of low-level actuator commands (e.g., vertical, horizontal, and rotational motions), which are then exposed to the digital twin through MQTT topics.

The highbay warehouse consists of nine slots, an entry and output point and a transport arm. This results in a payload being able to be in one of the nine slots, the arm or in the intake. The sorting stations is able to recognize three different colors, green, red and yellow in its intake. The outlets have push mechanisms to move it down the ramps. There are conveyors transporting the payload between the intake and the push arms. The multiprocessign station consists of

an intake, a turning table, and a oven module. The turntable is able to rotate in 270 degrees with three stations. One for processing the payload, one to recieve the payload from the intake and one for transporting to the oven module with a vacuum gripper. The oven consists of a door, a LED lamp and a tray that moves paylods in and out. The conveyor belts consist of a belt and two sensors, as light barrier at the ends of the belt to recognize any payload.

Each machine may show deviations that are machine specific. As an example the vacuum gripper shows a slight deviation due to a machine specific motion profile. They commonly are modeled as trapezoids or S curves [7]. The simulation in unity as depicted in Figure 1 reflects the machine control of the physical machine by providing each axis as a boolean value to be moved. The recognization of the payload happens here with simulated light barriers. These are realized using lines that provide a value upon collision with a payload. Each machine itself is composed of axis motors for each axis. The vacuum module is a unit itself. The conveyor belt can be taken as a belt connected to motors for the movement. While the physical system show clear structures, the virtual model contains multiple objects, which are connetcted through a reference as a whole entity to be moved. On a shop floor there are multiple roles [5]. The standard defines roles such as the maintenance Technician, Supervisor, Engineer, and Manager roles alongside with their competencies. For our fischertechnik example we keep a simpler setup. The Engineers and manager roles are provided by supervisors defining use cases of the production line. The supervisor must be a person knowing the production processes. Finally the maintenance technician is responsible for emergency exits, and fixing physical machines when encountering obstacles, such as a machine getting stuck.

2.3. Twinable Properties of Physical Modules

The properties that can be twinned cover the complete set of capabilities exposed by each physical station, including both (i) raw signals as used by the control software and (ii) derived, preprocessed properties for ease of use. Table **??** summarizes the mapping between hardware ports and the corresponding MQTT topics used in our digital twin system.

The vacuum gripper provides three degrees of freedom: a vertical axis, a horizontal axis, and a rotational axis. Each axis can be commanded via dedicated actuator signals (e.g., `move-vertical-up/down`, `move-horizontal-forward/backward`, and `move-rotation-clockwise/counterclockwise`), while reference switches and encoder impulses report the axis state. For instance, the vertical axis publishes encoder impulses on `motor-vertical-enc-1` and `motor-vertical-enc-2` and exposes a preprocessed integer position via the virtual topic `motor-vertical-pos`. In addition, the gripper includes pneumatic equipment that is controlled via compressor and valve commands (e.g., `enable-compressor` and `enable-valve`). We further model a suction property as a normalized value in [0, 1]; in more complex configurations, additional gripper-specific states (e.g., multiple tools or richer pneumatic models) could be incorporated.

The conveyor belt exposes two directions of motion and a small set of sensors. Concretely, it is commanded through `move-conveyor-forward` and `move-conveyor-backward`, while phototransistors (e.g., at the feed and swap stations) publish Boolean detection signals (cf. Table **??**). The sorting line extends this setup by adding sensing and actuation for quality decisions: a color sensor publishes an integer reading (topic `color-sensor`), and multiple light barriers delimit the measurement and routing sections (e.g., `light-barrier-inlet` and `light-barrier-behind-color-sensor`). Sorting is executed via three pneumatic ejectors controlled by separate valve signals (e.g., `enable-valve-first-ejector` to `enable-valve-third-ejector`).

The most complex station is the multi-processing station, which combines a turntable, a heating module, and a vacuum-based transport mechanism. Its turntable can be rotated to discrete positions (e.g., vacuum, belt, saw), controlled via `move-turn-table-clockwise` and `move-turn-table-counterclockwise`, and monitored through reference switches (e.g., `ref-switch-rotation-at-vacuum` and `ref-switch-rotation-at-belt`). The heating module provides a mocked temperature value and exposes an LED indicator (`enable-oven-light`) as a visible proxy of the heating process. Material flow is finalized through an output conveyor driven in two directions (topic `move-converyor-forward`). Additional sensors, such as light barriers at the oven and conveyor end, provide feedback for process coordination.

Finally, the high-bay warehouse comprises nine storage bays, an elevator mechanism, and an instrumented exit. Its motion is controlled via horizontal and vertical actuators (e.g., `move-arm-to-rack`, `move-arm-to-conveyor`, `move-vertical-up/down`) and a cantilever (e.g., `move-cantilever-forward/backward`). Encoder impulses and virtual position topics (e.g., `horizontal-enc-`, `vertical-enc-`, `horizontal-pos`, `vertical-pos`) allow the twin to track the current pose. Based on these signals and the exit sensors (e.g., `light-barrier-inside/outside`), the warehouse maintains an internal occupancy model of which bays are currently filled.

Inter-station coordination requires a digital twin system that integrates these heterogeneous signals into a consistent digital shadow. Each RevPi publishes raw I/O states via MQTT (mapped from the control-level ports in Table **??**); downstream components can then compute derived properties (e.g., discrete positions from encoder impulses) and expose them as higher-level, machine-readable twin properties.

3. Reference Architecture / Glossary (Andreas)

ca as in the book but dramatically condensed

4. Digital Twin Use Cases

In this section, we present three use cases for digital twins built on the Fischertechnik production line, which previous sections described as the actual system. Each use case presents a problem from the production domain, introduces a set of user stories and requirements for the digital twin, presents a solution architecture, and discusses an implementation of the digital twin.

With these use cases, we aim to demonstrate the specific problems digital twin engineers must solve when creating a digital twin for the manufactoring domain. Furthermore, we illustrate how practitioners can adapt the reference architecture from [11] to fit specific scenarios. We also discuss how effectively this architecture resolves common problems within the manufacturing domain. Additionally, future researchers can use the provided user stories and requirements to compare alternative development methodologies and architectures for digital twins.

The use cases are ordered from least complex to most complex, such that parts of the architecture from the earlier use cases can be reused in the later once. Additionally, the use cases are structured such that the digital twins of production line can run in parallel. Thus, they serve as base for a discussion of federation and interaction of multiple digital twins.

4.1. Use Case 1: Monitoring for Maximum Energy Consumption

4.1.1. Motivation

Regulations or availabilty can put a limit on the peak consumption of a resource, e.g., the maximum power a factory can use at any one time. In a factory, exceeding this availability can stop production , and failing to comply with regulations may lead to fines or shutdown. Thus, the digital twin should enable measurement, logging, and management of scarce resource usage.

In this example, we focus on a production line connected to a smart energy grid, which imposes a dynamic limit on the total power usage. To ensure this constraint holds, the consumption data of the individual machines must be measured in the physical twin and synchronized with the digital twin. The digital twin then aggregates the consumption data and compares it with the target value. To enable long-term auditing, both the individual and aggregated measurement series are permanently stored in digital shadows. To prevent overuse, the digital twin must send control signals back to the physical twin, for example to temporarily stop individual machines until sufficient energy budget is available again.

4.1.2. Stakeholders and User Story

Stakeholders. The main stakeholders are the *factory manager* and the *operator*.

User stories. As a *operator*, I want to

- **US1.1** monitor the current energy consumption of each machine because I want to detect anomalies.
- **US1.2** monitor the current aggregated energy consumption of the production process because I want to plan next production steps.
- **US1.3** be notified if the current energy consumption of the production process is exceeding the limit set by the smart energy grid because I want to avoid a shutdown.

As a *factory manager*, I want to

- **US1.4** avoid exceeding the energy consumption limits imposed by the smart grid because I want to avoid shutdown of the production.

Fulfillment criteria/Requirements Next, the user stories are converted into smaller, independent requirements, which form that bases for an solution implementation.

RQ1.1 The energy consumption of each machine must be measured and logged.

RQ1.2 The current energy consumption of the overall production process must be calculated and logged.

RQ1.3 The energy consumption limit must be obtained from the smart grid.

RQ1.4 The difference between the energy consumption limit and the total production consumption must be calculated and logged.

RQ1.5 The system must provide a real-time visualization of the energy consumption of each machine, the aggregated production consumption, and the difference between the limit and the aggregated consumption.

RQ1.6 The system must notify the operator and the factory manager if the aggregated energy consumption approaches or exceeds the imposed limit.

US1.1 necessitates the measurment of energy usage on the machine level, leading to **RQ1.1**. Since the operator manual needs to manualy check the current and historical state of this consumption, a dashboard needs to be present(**RQ1.5**). Furthermore, **US1.2** requires that the operator can observe the current aggregated energy consumption of the production process. This directly implies the continuous aggregation of machine-level measurements (**RQ1.1**) into a process-level metric, resulting in **RQ1.2**. In addition, planning future production steps necessitates access not only to instantaneous values but also to historical trends, which is covered by the logging aspect of **RQ1.2** and the visualization requirement in **RQ1.5**.

US1.3 introduces the constraint that the system must react to externally imposed energy limits from the smart grid. Consequently, the system must be able to retrieve and update the current limit dynamically, which is formalized in **RQ1.3**. To determine whether the limit is being approached or exceeded, the system must continuously compare the aggregated consumption with the limit, leading to **RQ1.4**.

Based on this comparison, timely notifications to the operator are required, which is specified in **RQ1.6**. The real-time aspect of the user story further reinforces the need for live visualization as defined in **RQ1.5**.

US1.4 from the factory manager perspective focuses on preventing production shutdowns caused by exceeding smart grid limits. This objective aligns with proactive monitoring and early intervention. Therefore, the notification mechanism in **RQ1.6** must not only trigger upon limit violations but also when the consumption approaches a critical threshold. Moreover, the continuous acquisition of the grid limit (**RQ1.3**) and the comparison logic (**RQ1.4**) are essential to support strategic decision-making at the management level.

4.1.3. Digital Twin Design to Fulfill the User Story

The user stories are solved through a digital twin of the fischertechnik production line. In a first step, we create a functional model of the machines and the production line using a MontiArc Component and Connector diagram [8], the result of which can be seen in Figure 4. Since the main focus is on the energy consumption, most aspects related to the production are left vague via underspecification. The top-level function *ProductionLine* gets material as an input and outputs the finished product. Additionally, it has an output port for the aggregated energy usage of the production line, working towards fulfilling **RQ1.2**. The material from the input is passed trough multipe transformations, such as the *Drilling* function, as well as functions to model the transport between the transformation steps. The transport components are necissary, since the physical components implementing the functions, such as a vacuum gripper, will also use energy. Each of the internal components measures and outputs its current energy usage as data via a sensor. All of these datapoints are fed into the *sum* component, which aggregates and forwards it to the output of *ProductionLine*. Thus, the basis for fulfilling **RQ1.2** also set.

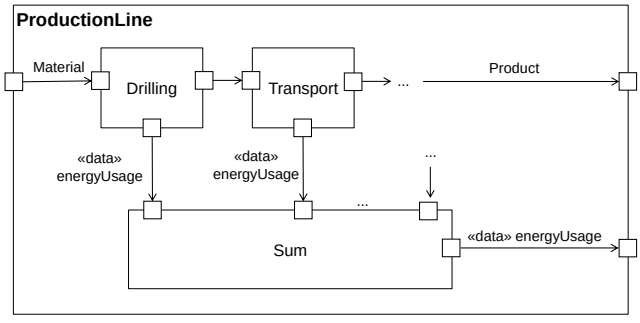


Figure 4: High level C&C model of the production line with focus on the energy consumption

From this architecture of the CPS, the first digital shadows to implement the user stories can be derived. The channels streaming data between the transformation steps and transport functions to the *Sum* function can be observed to receive the digital shadows of the energy consumption for each machine(**RQ1.1**). Additionally, the maximum energy provided by the smart energy grid needs to be saved in a digital shadow, in relation to **RQ1.3**.

The channel between the *Sum* and the top-level function represents the shadow of the total energy consumption(**RQ1.2**). Since it is an aggregation of the energy consumption shadows of each machine, it needs to be calculated by a service, which we will call *shadow aggregator* from now on. The last shadow of this digital twin is the difference between the current usage and the limit, from **RQ1.4**, and it is also calculated by the same *aggregator* service.

Based on these shadows, the other services of the digital twin can be designed. To fulfill **RQ1.5**, all of the digital shadows need to be accessible in a dashboard such that the *operator* and *factory manager* can manually inspect and plan based on energy consumption and availability. As an easy-to-understand visual representation of the shadows, multiple line charts are presented on the dashboard. To allow the focus on a subset of the shadows, the visibility of the charts is dynamically toggleable.

To fulfill requirment **RQ1.6**, a notification service is added, which observes the difference between the available and used energy and sends notifications, such as emails or push notifications, to the *operator* and *factory manager*. These messages are sent when configured limits are exceeded.

4.1.4. Example Architecture in the Fischertechnik Demonstrator

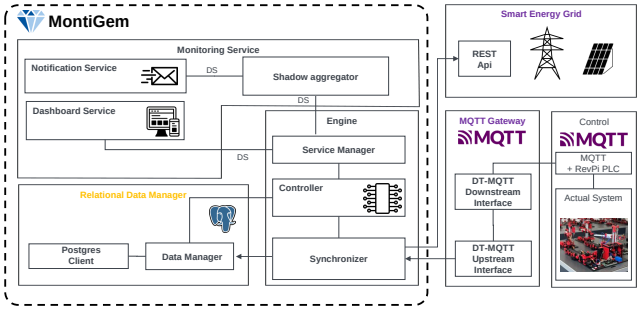


Figure 5: Architecture of the digital twin for the energy monitoring use case

Make a Compone Digram o this

For the implementation in the fischertechnik demonstrator, the architecture displayed in Figure 5 is used. The actual twin, the fischertechnik production line, is controlled by RevPI PLCs, and their control software contains an MQTT gateway, which can be used to send control commands to or receive data from the production line. All sensor data and the current state of the machines are sent proactively by the PLCs to the MQTT broker, which is not displayed in the figure. The gateway of the digital twin also uses MQTT, and its *DT-MQTT Downstream Interface* is subscribed to all relevant energy sensor traces from the actual system. The *DT-MQTT Upstream Interface* translates the data types used in MQTT into the data types of the digital twin.

Afterwards, all data of the physical systems is passed to the *Synchronizer* of the DT-Engine, which ensures that

the incoming shadow data is synchronized into the data management and to the services. Additionally, it polls the REST interface of the smart grid to retrieve the currently available energy. The *Data Manager* serializes data into a Postgres database and provides a cache for recently used data.

The *Service Manager* initializes and connects the services of the system. The *Shadow Aggregator* observes the incoming digital shadows via the *Service Manager* and creates the two processed shadows, the aggregated energy usage and the difference to the available energy.

The *Dashboard Service* accesses the raw and processed digital shadows via the Data Manager and provides a real-time visualization of machine-level consumption, aggregated usage, and the remaining energy budget. It offers interactive line charts and configurable filters so that operators and factory managers can focus on specific machines or time intervals for detailed analysis.

The *Notification Service* subscribes to the aggregated consumption and difference shadows and continuously evaluates them against configurable warning and limit thresholds. Whenever the remaining energy budget falls below a defined margin or the limit is exceeded, the service automatically dispatches alerts, such as emails or push notifications, to the relevant stakeholders.

Together, these three services comprise the overall monitoring service of the digital twin. Finally, the *Synchronizer* and *Service Manager* are orchestrated by the *Controller*.

4.1.5. Example Implementation in the Fischertechnik Demonstrator

To implement this architecture, the MDE generator framework MontiGem [3] is used, which generates a full web application comprised of a backend, database, and frontend. To define the data structure used across all three of these components, class diagrams are used. In addition, GUI-DSL models are used to define the pages of the frontend. Handwritten interfaces to and from other systems are written in Java with the help of Spring Boot. Java can also be used to add behaviour to gui pages and implement additional services.

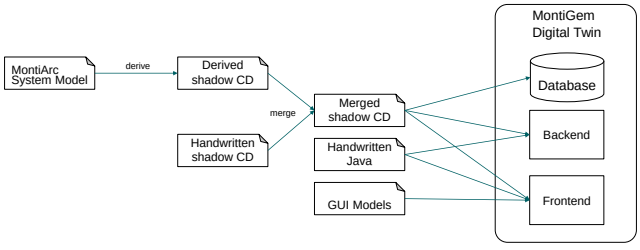


Figure 6: Generation of the Digital Twin using MontiGem

The artifacts used as inputs in generation process of the digital twin can be seen in Figure 6. To be usable in the rest of the system, the digital shadows have to be defined in the class diagram. In this simple use case, all values are saved as a time series of doubles. While the shadow from the smart power grid is modelled manually, all other shadows can automatically derived from the functional model of the CPS. A DT specific generator reads in the MontiArc model, extract the function structure, and converts all connectors between components into a class diagram of the digital shadows. This diagram is automatically merged with the hand written shadow definition and used to genereate the base data structure of the MontiGem application.

To fill the digital shadows with data, the gateways to the external systems need to be implemented. By augmenting the MontiArc model via stereotypes, the connectors used as a data source for each shadow can be defined. Each outgoing port related to energy measurment of the transformation and transport steps is enriched with the MQTT topic used to broadcast the sensor values. A generator is then used to derive the transformation from MQTT data types into the DT Datatypes. Thus, the *DT-MQTT Upstream Interface* is implemented. The *DT-MQTT Downstream Interface*, which connects the DT to the MQTT broker, needs to be manualy implement and configured.

The generated MontiGem application contains a mechanism to automatically synchronize data changes into the data base and connected frontend clients. This infrastructure takes on the role of the *Synchronizer*. A hand-written extension to the *Synchronizer* is made, which polls the REST api of the smart energy grid and writes the returned data into the appropriate digital shadow.

The *Data manager* is automatically generated from the merged class diagram describing the digital shadows, including a schema for the Postgres database, all DAOs needed to translate between the data base types and the DT types, and a caching layer for often used data. The *Postgres Client* is provided as a configurable runtime component.

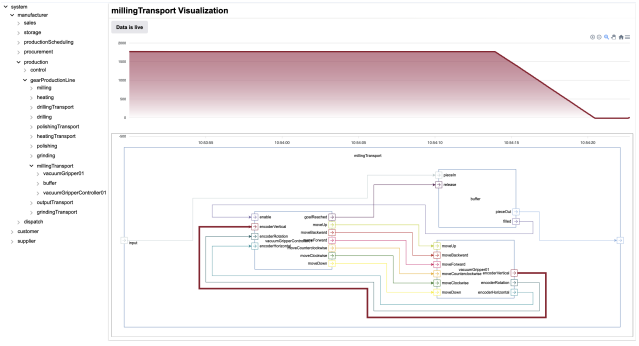


Figure 7: Dashboard Service: Visualization of the energy usage digital shadows

4.2. UC2:Flexible Replanning

4.2.1. Motivation

Unexpected failures or temporary unavailability of critical components in a factory can cause production disruptions and unplanned downtime. Since production steps are interdependent, a local disruption in a component can prevent the

next steps to be executed as planned, even if the rest of the production line is operational. To reduce such disruptions, the digital twin must be able to flexibly replan the factory's operation by choosing an alternative way to achieve the same production goal when the nominal path is disrupted.

In the fischertechnik factory, the production process relies on successive transfer and forwarding actions to move a workpiece between the factory's stations. We consider a scenario where a component failure prevents the nominal path from progressing as expected. The digital twin monitors the execution to detect such faults, logs when and where they happened and switches to an alternative execution path that advances the workpiece thus avoiding unnecessary downtime.

4.2.2. Stakeholders and User Stories

Stakeholders. The main stakeholders are the *operators* (monitoring execution), *the factory manager* (minimize downtime, meet production targets) and

User Stories. As a *maintenance engineer*, I want to

- **US1** be notified when the digital twin to detect when the nominal production path is disrupted , so that I can intervene for maintenance operations.

As a *factory manager*, I want to

- **US2** analyze when and where replanning was triggered, so that I understand recurring disruptions and evaluate the chosen strategies.
- **US3** minimize downtime so that the production targets can be met.
- **US4** analyze records of component downtime and maintenance duration so that I can make informed reinvestment decisions.

Fulfillment criteria/Requirements

RQ 1 The system must continuously monitor the progress of the nominal production path during operation and keep track of the current execution state.

RQ 2 The system must detect disruptions when the expected progress condition for the current step is not met

RQ 3 The system must record the payload of the disruption including the timestamp, the current affected step and the active nominal path.

RQ 4 The system must select an alternative execution path when a discrepancy is detected, to keep the production cycle running with the intended objective.

RQ 5 The system must record the alternative strategies and when they were applied, so that recovery actions can be analyzed.

RQ 6 The system must notify the maintenance engineer when a disruption in the nominal path is detected, including information about the affected component.

RQ 7 The system must record the duration of each disruption, from disruption to recovery, in order to quantify downtime.

RQ 8 The system must compute indicators, such as mean time between failures, mean time to repair for each component, in order to quantify downtime impact and justify component replacement.

US1 requires data to detect deviations and to support intervention. This implies maintaining a continuously updated representation of the current execution state and the expected nominal progression at that state (**RQ1** and **RQ2**), and capturing the deviation context that can be communicated in the notification, including when and where it occurred and the affected component (**RQ3** and **RQ6**). **US2** requires traceability data to support analysis of replanning, which means the system must persist historical records that locate each deviation in time and along the nominal path and must also persist which alternative fallback strategy was selected and when it was applied, as captured by **RQ3** and **RQ5**. **US3** focuses on keeping the production cycle running to reach the intended objective when the nominal path is disrupted. The system then must react to a detected deviation by choosing an alternative strategy, rather than stopping the production cycle (**RQ4**). **US4** requires downtime measurements for each component comparable over time. This necessitates recording, for each deviation, the affected component, the deviation's start timestamp and the recovery timestamp, so each disruption duration can be computed and then aggregating these duration per component in line with **RQ7**, while also providing the required inputs to compute component reliability indicators in **RQ8**.

4.2.3. Digital Twin Design to Fulfill the User Story

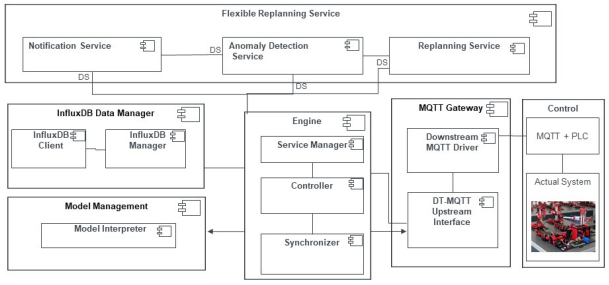


Figure 8: Digital Twin Architecture for the flexible replanning use case

Make a Compone Digram o this

Make a State Cha

We model the production cycle as the progression of a workpiece across the different stations of the FT demonstrator, using a SysML v2 state machine. In our scenario, a conveyor belt (CB) failure disrupts the expected progression and triggers a predefined recovery path that reroutes the workpieces using nearby and available resources to meet the production target. Once the maintenance engineer restores the CB, the system can return to the nominal production path.

To implement this behavior, the digital twin continuously observes the PLC's incoming feedback and checks it against the expected workpiece progression derived from the state machine. The connection between the DT and the CPS is provided by an MQTT gateway at the CPS level, which continuously publishes feedback to the MQTT broker. The *DT-MQTT Downstream interface* subscribes to the relevant MQTT topics. The *DT-MQTT Upstream Interface* then translates received payloads and maps them onto the DT's internal variables and types, consistent with the model attributes used by the DT. The resulting feedback is then forwarded to the *Synchronizer*, which then routes them to the data management layer, and makes it available to the model management layer and to the services.

The behavioral model is handled by the model management layer, where the *Model Interpreter* loads the SysML v2 state machine and parses the model to expose the expected workpiece progression along the nominal and recovery path, along the conditions under which the transitions occur. The data management layer records timestamped feedback and relevant events in the *InfluxDB manager*.

The *Service Manager* initializes and coordinates the different services and provides them access to both the interpreted model information and the feedback data. The *Anomaly Detection Service* compares the expected progression derived from the state machine with the observed progression of the CPS and flags an anomaly when a disruption is detected. The *Notification Service* then alerts the maintenance engineer so they can intervene. The *Re-planning Service* switches execution from the nominal path to the recovery path and later returns to the nominal path when the maintenance engineer confirms that the faulty component is operational again.

4.2.4. Example Implementation in the Fischertechnik Demonstrator

To implement the architecture (8), we configured the digital twin using the SysML v2 model, which provides both structural and behavioral descriptions of the CPS.

From the structural models, we generate (i) the database schema used by the *InfluxDB Manager*, and (ii) the *DT-MQTT Upstream Interface*. On the digital twin side, this interface exposes the set of modelled data types that the digital twin expects and maps each expected DT type to its corresponding MQTT topic while the *DT-MQTT Downstream Interface* captures the MQTT topics published by the CPS.

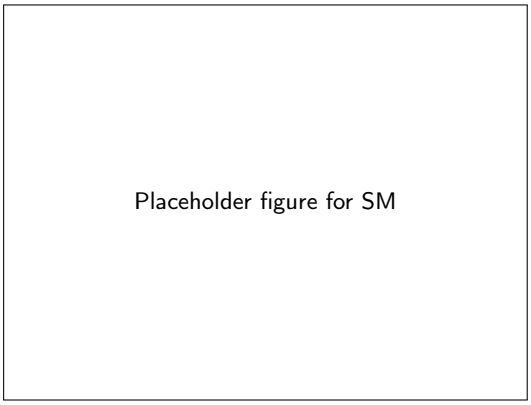


Figure 9: (WIP)State Machine of the Workpiece's Location on the FT Demonstrator

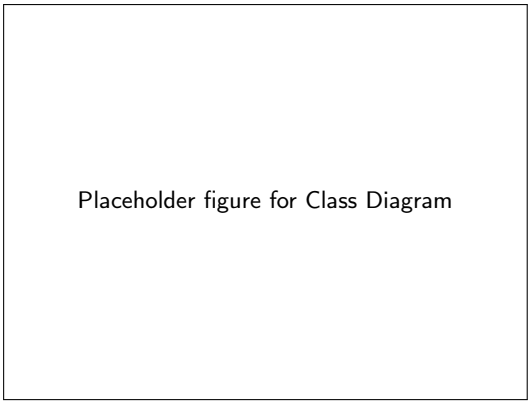


Figure 10: Class Diagram of the components needed to build the digital twin for UC2

For the behavioral models, we configure the *Anomaly Detection Service* with the state machine, which continuously compares the expected modelled progression to the observed behavior of the CPS. We also configure the *Replanning Service* from the same models, so that a detected disruption triggers a transition to the predefined fallback strategy.

The notification service is a GUI intended for the maintenance engineer to be informed that there has been a disruption of the nominal path, along with the identified faulty component and the exact location where the workpiece is blocked, so they can intervene. On the same GUI, the maintenance engineer can confirm that the maintenance has completed, which allows the production cycle to return to the nominal production path. –todo:: transition to nominal pat, the switch from the fallback strategy SM to the nominal one

4.3. Use Case 3: Predictive Maintenance

4.3.1. Motivation

Highly flexible and frequently reconfigured production systems are a response to fast-changing markets, but they also amplify the need for continuous visibility into machine and process state. In such settings, maintenance can no

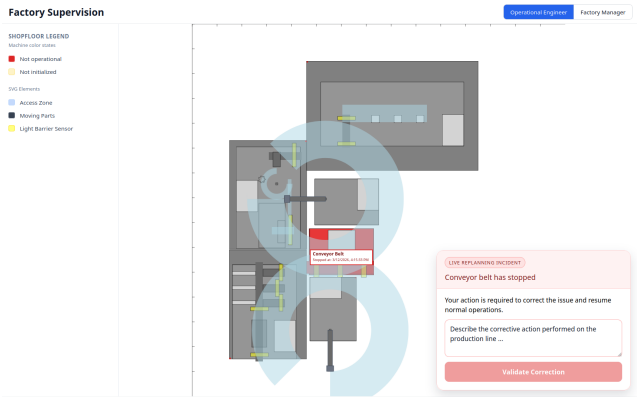


Figure 11: Digital Twin Notification Service GUI

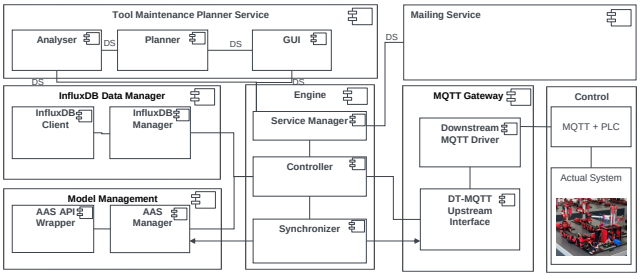


Figure 12: Overview of the Digital Twin Architecture for a Predictive Tool Maintenance Service

longer rely on purely reactive interventions; instead, data-driven predictive maintenance aims to anticipate failures early, reduce unplanned downtime, and stabilize output quality by intervening before degradation becomes irreversible [10, 12]. Prior work has already demonstrated that self-adaptive digital twins can use operational signals to reason about quality-related indicators and explore parameter spaces under varying conditions (e.g., SDMflex) [1, 2].

In our demonstrator, degradation manifests as increasing deviations in motor speed and cycle timing under comparable operating conditions. Such deviations are commonly driven by cumulative wear and load-dependent effects in motion systems; accuracy degradation is widely attributed to wear mechanisms that accumulate with usage and loading [4, 13]. Moreover, the operating load (here: payload weight) has a direct impact on the dynamic response of conveying systems and thus on observable signals such as transient speed profiles and timing [9]. Consequently, the digital twin monitors speed and timing deviations, predicts whether the trend will reach a failure threshold, and triggers maintenance and recalibration actions before the physical system breaks down.

4.3.2. Stakeholders and User Story

Stakeholders. Primary stakeholders are (i) maintenance engineers, who plan and execute interventions, (ii) operators, who observe anomalies and apply operating procedures, (iii) production planners, who coordinate downtime windows, and (iv) quality managers, who relate process stability to product quality and scrap rates.

User stories. **US1** As a maintenance engineer, I want the system to monitor operational speed deviations of the drive motor under different payload weights and duty cycles, so that I can detect early signs of wear or overload and schedule maintenance before a breakdown occurs.

US2 As an operator, I want the system to notify me when abnormal motor behavior occurs and provide guidance on the affected component and recommended handling procedure, so that I can react quickly and operate the line safely.

US3 As a production planner, I want the system to provide predicted maintenance windows and expected intervention durations, so that I can coordinate maintenance activities with production schedules and minimize disruption.

US4 As a quality manager, I want the system to relate motor degradation and maintenance events to product quality indicators and scrap occurrences, so that I can assess whether equipment degradation affects production quality.

Fulfillment criteria/Requirements.

RQ 1 Continuous logging. The system must continuously record time-series data for the drive motor, including at least motor speed and sensor arrival times for each production cycle, together with timestamps and a unique cycle or part identifier.

RQ 2 Context capture. The system must record the operational context for each production cycle, including payload weight (e.g., normal or increased), operating mode (e.g., idle, normal load, high load), number of start–stop cycles, cumulative operating hours, and production counts which should be described during desing time.

RQ 3 Baseline and anomaly detection. The system must derive an expected speed profile for normal operation and detect anomalies when the observed speed deviates from the expected profile by more than a configurable threshold for the same payload weight and duty cycle.

RQ 4 Actionability for maintenance. The system must notify the maintenance engineer when a failure-prone trend is detected and provide a recommended maintenance window together with a concrete intervention, such as recalibration, reset procedures, or mechanical inspection.

RQ 5 Actionability for operators. The system must notify the operator when abnormal behavior is detected and provide information about the affected component, the anomaly type, and the recommended immediate handling procedure.

RQ 6 Planning support. The system must estimate the expected duration and urgency of a recommended intervention and make this information available for production planning purposes.

RQ 7 Quality correlation. The system must relate detected anomalies and maintenance events to production outcomes, including quality indicators and scrap, so that the effect of equipment degradation on product quality can be analyzed.

RQ 8 Historical analysis and model support. The system must persist anomaly cases, maintenance events, and associated context data for historical analysis and for the training and evaluation of predictive maintenance models. At least N synthetic anomaly scenarios shall be supported and labeled as "failure-prone" cases.

The maintenance engineer user story requires the system to identify abnormal motor behavior under varying production conditions and to translate these observations into actionable maintenance decisions. This requires continuous operational data acquisition (**RQ 1**), contextualization of each cycle (**RQ 2**), anomaly detection against expected behavior (**RQ 3**), and the derivation of concrete maintenance actions (**RQ 4**).

The operator user story requires timely visibility of anomalies during plant operation. This means the system must not only detect abnormal behavior (**RQ 3**) but also communicate the affected component and suitable handling instructions to support safe and immediate intervention (**RQ 5**).

The production planner user story requires that predictive maintenance results can be incorporated into scheduling decisions. This implies that the system must transform anomaly and maintenance findings into planning-relevant information, especially intervention urgency and expected duration, as specified in **RQ 6**.

The quality manager user story requires that equipment degradation can be assessed in relation to product quality and scrap. This necessitates linking anomaly and maintenance records to production outcomes and quality indicators, as required by **RQ 7**. Historical persistence of these cases is further required for retrospective analysis and model improvement, as captured in **RQ 8**.

4.3.3. Digital Twin Design to Fulfill the User Stories

To fulfill the user stories introduced above, we implement the predictive maintenance capability shown in Figure 12 on top of the lightweight digital twin architecture introduced in the previous subsection. The cyber-physical system, consisting of PLC-controlled machinery and MQTT-based communication, publishes operational signals to an MQTT broker. The digital twin subscribes to these signals, synchronizes them into its internal representation, persists them, analyzes them, and triggers role-specific responses for maintenance engineers, operators, production planners, and quality managers.

In the concrete realization, the digital twin engine comprises a controller, a synchronizer based on explicit signal-to-model mappings, and a service manager. The MQTT-based ingestion and synchronization pipeline provides the basis for continuously acquiring and structuring operational data such as motor speed, sensor arrival times, and cycle identifiers, thereby fulfilling **RQ 1**. The synchronizer further enriches incoming signals with production-related context, such as payload class, operating mode, start–stop frequency, cumulative operating hours, and production counts, thereby addressing **RQ 2**. In this way, the digital twin maintains a continuously updated and context-aware representation of the monitored asset and its behavior.

For persistence, the system stores structured asset and model information in an Asset Administration Shell and time-series operational data in an InfluxDB instance. The InfluxDB back end provides the historical storage required for continuous logging and retrospective analysis, directly supporting **RQ 1** and **RQ 8**. The Asset Administration Shell captures asset-related metadata, component descriptions, and contextual information relevant to interpreting operational deviations, thereby contributing to **RQ 2**. Both persistence back ends are accessed through dedicated management components that expose minimal read and write interfaces to the digital twin engine.

On top of this data foundation, the service manager hosts reusable service components that realize the analysis, planning, notification, and reporting functionality required by the different stakeholders. An *Analysis* component evaluates the logged speed and context data against expected baseline profiles in order to identify anomalies and failure-prone trends. This component therefore implements the core functionality required by **RQ 3**. In addition, the same analysis functionality supports the persistence and labeling of anomaly cases for later evaluation and model improvement, contributing to **RQ 8**.

A *Maintenance Planner* service realizes the planning-oriented part of the capability. Based on the detected anomaly patterns, it derives maintenance recommendations, proposes concrete interventions such as recalibration or inspection, and estimates suitable maintenance windows and expected intervention durations. In this way, it fulfills **RQ 4** for the maintenance engineer and **RQ 6** for the production planner. The planner thus transforms analytical findings into actionable support for scheduling and intervention preparation.

To support operators during runtime, the service layer further provides a notification mechanism that communicates detected abnormal behavior together with the affected component and recommended immediate handling procedures. This realizes **RQ 5** and ensures that anomalies are not only detected analytically but also made operationally visible in a form that supports safe and timely reaction on the shop floor.

Finally, the digital twin exposes historical and aggregated data for quality-oriented assessment. By linking anomaly events and maintenance records with production outcomes such as quality indicators and scrap occurrences, the system supports the quality manager in analyzing whether equipment degradation affects product quality. This realizes **RQ 7**. Since these records are also retained over time, the digital twin supports longer-term retrospective analysis

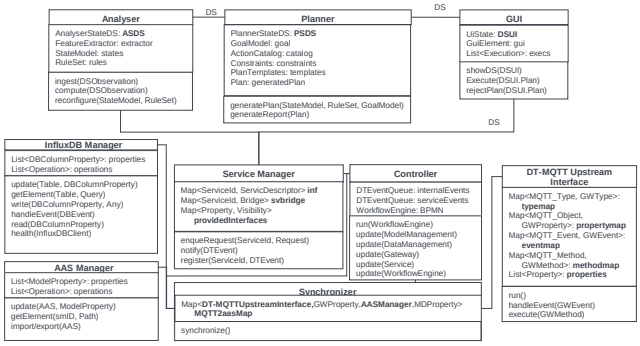


Figure 13: Class diagram of the minimal components to build a digital twin for predictive maintenance.

across maintenance, planning, and quality perspectives, again reinforcing **RQ 8**.

Overall, the digital twin design fulfills the user stories through a layered realization of the requirements: data ingestion and synchronization address **RQ 1** and **RQ 2**, analysis addresses **RQ 3**, maintenance planning addresses **RQ 4** and **RQ 6**, operator notification addresses **RQ 5**, and historical correlation with production outcomes addresses **RQ 7** and **RQ 8**. In this way, the digital twin does not merely monitor the asset, but provides an integrated basis for predictive maintenance, operational response, production coordination, and quality assessment.

Digital twin classes The digital twin classes as depicted in Figure 13 consist of the following classes:

- **DT–MQTT Upstream Interface.** Manages type/object/event/method mappings (typemap, propertymap, eventmap, methodmap) plus the available properties, and exposes the runtime entry points for MQTT-based DT I/O (run, handleEvent, execute).
- **Service Manager.** Orchestrates services via descriptors and bridges, manages provided interfaces/visibility, and coordinates requests and DT events through internal and service event queues (enqueueRequest, notify, register).
- **Synchronizer.** Owns the end-to-end synchronization logic (e.g., MQTT↔AAS property mapping), executes synchronization cycles, and propagates updates to model, data, gateway, service, and workflow components (synchronize, update(...)).
- **Controller.** Runs and controls workflow execution by driving a workflow engine (BPMN) through the main control entry point (run(WorkflowEngine)).
- **AAS Manager.** Manages AAS properties and operations, supports updating AAS elements and path-based access, and handles AAS import/export (update, getElement, import/export).
- **InfluxDB Manager.** Encapsulates time-series persistence around DB column properties and operations, supporting reads/writes, event handling, and client health checks (write, read, handleEvent, health).
- **Analyser.** Ingests observations into an analysis digital shadow, extracts features, computes states/results, and supports reconfiguration of the state model and rule set (ingest, compute, reconfigure).
- **Planner.** Generates a plan from the current state/rules and goals, using an action catalog plus constraints and templates, and can produce a report from the plan (generatePlan, generateReport).
- **GUI.** Presents the digital shadow UI state, visualizes it, and lets a user approve or reject the proposed plan (showDS, Execute, rejectPlan).

4.3.4. Models and Data Used in Analysis & Planning

Predictive maintenance requires linking observed signals to the context that explains them. We therefore represent the machine state with (i) operational time series, (ii) per-cycle aggregates, and (iii) cumulative usage indicators commonly used to estimate how much "life" has been consumed (e.g., hours, cycles, start–stop counts) [12, 10].

Operational and usage data model. For each cycle c, we store:

- **Signals:** motor speed $v(t)$; sensor arrival timestamps $\tau_s(c)$; optionally current/temperature/vibration (if available).
- **Context:** payload class $w(c) \in \{\text{normal}, \text{increased}\}$; duty-cycle/mode $m(c)$; start–stop count $\sigma(c)$.
- **Cumulative usage:** operating hours $H(c)$; production count $P(c)$.

Load-dependent effects are explicitly modeled by conditioning the baseline on payload and mode; this mirrors the general observation that loading affects dynamic behavior and thus the expected signal profile [9, 13].

Synthetic failure labels for evaluation. To enable repeatable experiments in the Fischertechnik setup, we create controlled anomaly scenarios by: (i) increasing payload weight (added mass), (ii) increasing friction (e.g., intentional mechanical resistance), or (iii) inducing mild misalignment. All cycles recorded under these conditions are labeled as "failure-prone" to evaluate anomaly detection and (simple) remaining-useful-life trend estimation.

4.3.5. Adaptation Logic and Maintenance Actions

The analysis component computes (a) per-cycle deviation features and (b) a trend estimate. Deviations are compared against a baseline conditioned on payload and duty cycle; if deviation grows persistently or crosses a threshold, the digital twin plans an intervention. This closes the loop from monitoring to actionable maintenance planning, consistent with MAPE-K-style digital twin integration [6].

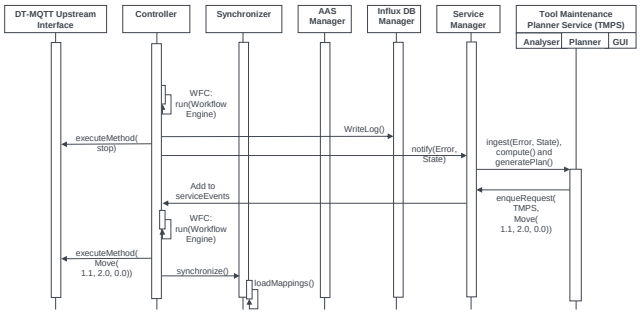


Figure 14: Dataflow in the digital twin when encountering an unknown state or anomaly that triggers analysis and planning.

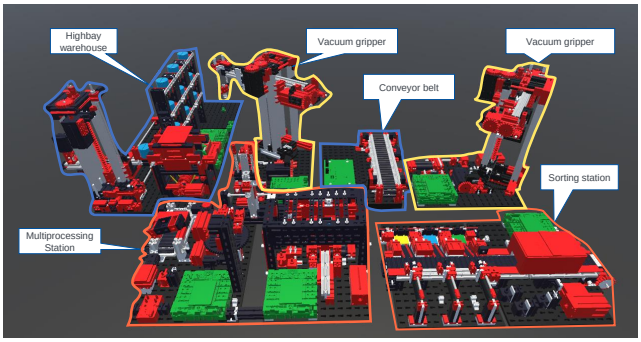


Figure 15: Visualization in User Interface of Demonstrator showing the current state and showing degradation of modules in UI.

Typical intervention policies.

- **Early warning:** deviation exceeds 40% for 3 consecutive cycles ⇒ notify maintenance engineer and operator.
- **Safe fallback:** deviation increases sharply under high load ⇒ reduce speed setpoint or limit routing/throughput.
- **Recalibration/reset:** if deviation indicates encoder drift or accumulated offset, execute a reset/re-homing procedure and log the effect (before/after comparison).
- **Planned maintenance:** if trend predicts threshold crossing within a horizon (e.g., next T cycles), propose a maintenance window for inspection and readjustment.

4.3.6. Example Implementation in the Fischertechnik Demonstrator

As illustrated in Figure 14, the runtime follows a closed-loop sequence: the Workflow Controller triggers synchronization and logging, executes a safe fallback action (e.g., `stop`), notifies the planner service with `Error` and `State`, and then executes the planner's parameterized action request (e.g., `Move(1.1, 2.0, 0.0)`) via the upstream interface.

Signals and MQTT topics. The CPS publishes station signals and process context via MQTT (broker as introduced previously). In line with the topic structure used throughout this paper, machine-related topics are published in a machine-specific namespace following the pattern `/${machine-id}/...`. For the predictive-maintenance use case, the digital twin subscribes to a combination of (i) station-level sensor/actuator topics and (ii) process-context topics.

Examples of station-level topics used in the UC3 storyline are:

- **Arrival / transfer events** (used to reconstruct cycle timing):
  - `/${machine-id}/light-barrier-inlet`
  - `/${machine-id}/light-barrier-behind-color-sensor`
  - `/${machine-id}/phototransistor-feed-station`
  - `/${machine-id}/phototransistor-swap-station`
- **Motion-related raw signals** (used to derive speed and movement behavior):
  - `/${machine-id}/motor-vertical-enc-1`, `/${machine-id}/motor-vertical-enc-2`
  - `/${machine-id}/motor-horizontal-enc-1`, `/${machine-id}/motor-horizontal-enc-2`
  - `/${machine-id}/motor-rotation-enc-1`, `/${machine-id}/motor-rotation-enc-2`
  - `/${machine-id}/pulse-counter` (e.g., sorting line)
- **Derived position signals** (published by the control layer after preprocessing):
  - `/${machine-id}/motor-vertical-pos`
  - `/${machine-id}/motor-horizontal-pos`
  - `/${machine-id}/motor-rotation-pos`
- **Relevant actuation/operating signals** (for contextualizing load and duty behavior):
  - `/${machine-id}/move-conveyor-forward`, `/${machine-id}/move-conveyor-backward`
  - `/${machine-id}/enable-compressor`
  - `/${machine-id}/enable-valve` (or station-specific valve topics)

In addition, this uses process-level context topics (published by the CPS/software layer) to associate each cycle with operating conditions and traceability metadata, e.g.:

- `/line/cycle/id` (cycle/part ID),
- `/line/context/payload` (`normal` / `increased`),
- `/line/context/mode` (`idle` / `normal` / `high`).

This combination allows the digital twin to reconstruct cycle timing from sensor events, derive motion-speed-related features from encoder/pulse signals, and evaluate deviations under comparable payload and operating modes.

Feature computation and anomaly detection. On each cycle c, the digital twin aggregates encoder-/pulse-based motion signals and sensor arrivals into cycle-level features. In particular, it derives a speed estimate $\bar{v}(c)$ from the encoder or pulse-counter streams (optionally supported by derived position topics) and combines it with cycle timing features. In particular, it derives

$$\Delta v(c) = \frac{\left|\bar{v}(c) - \bar{v}_{\text{base}}(w(c), m(c))\right|}{\bar{v}_{\text{base}}(w(c), m(c))}, \quad \Delta\tau(c) = \tau_{s2}(c) - \tau_{s1}(c),$$

and flags an anomaly if $\Delta v(c) > X$ (and/or timing residuals exceed a threshold) under the same payload/mode class.

Analysis–planning–execution loop. When an anomaly (or unknown state) is detected, the Workflow Controller (WFC) initiates the loop shown in Figure 14: (i) it calls the Synchronizer (`synchronize()`) which loads the required AAS mappings (AAS Manager: `loadMappings()`) and persists a trace entry (InfluxDB Manager: `WriteLog()`); (ii) the DT–MQTT upstream interface immediately enforces a safe fallback (e.g., `executeMethod(stop)`); (iii) the Service Manager publishes a structured notification containing `Error` and `State` (`notify(Error, State)`); (iv) the maintenance planner service (TMPS) ingests this information (`ingest(Error, State)`), computes a decision (`compute()`), and produces an executable plan (`generatePlan()`); (v) the Service Manager enqueues the resulting parameterized action request into `serviceEvents` (e.g., `enqueRequest(TMPS, Move(1.1, 2.0, 0.0))`); (vi) the WFC subsequently executes the queued request through the upstream interface (e.g., `executeMethod(Move(1.1, 2.0, 0.0))`).

Outputs. The implementation produces three artifacts aligned with Figure 14: (i) operator-facing alerts via notification (dashboard/event log) as in Figure 15, (ii) persistent anomaly traces and cycle summaries in the time-series database for offline learning (e.g., regression over deviation trends), and (iii) an executable recovery/maintenance action (e.g., inspect, recalibrate/reset, schedule downtime, or a concrete motion/stop command) emitted as a queued service request and executed by the WFC.

5. Lessons Learned

6. Discussion (Judith)

7. Conclusion and Perspectives (Benoit)

TBC. (a kind of call for actions) @Benoit

scuss
ues with
ning in
allel

mposing
DTs
ether?
hat are
ne im-
cations?

Acknowledgements

Funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) - Model-Based DevOps - 505496753. Funded by the Agence Nationale De La Recherche (ANR) - France. Website: https://mbdo.github.io.

CRediT authorship contribution statement

MBDO Dream Team: . : .

References

[1] , . Flexible sdm through continuously quality-aware digital twins (sdmflex). Project website, University of Stuttgart. Accessed 2026-01-27.

[2] , . Sdmanu4 – sdmflex. Innovation Campus Mobility of the Future (ICM) project page. Accessed 2026-01-27.

[3] Buschhaus, C., Gerasimov, A., Kirchhof, J.C., Michael, J., Netz, L., Rumpe, B., Stüber, S., 2024. Lessons Learned from Applying Model-Driven Engineering in 5 Domains: The Success Story of the MontiGem Generator Framework. Science of Computer Programming 232, 103033. URL: https://www.se-rwth.de/publications/Lessons-learned-from-applying-model-driven-engineering-in-5-domains-The-succes pdf, doi:10.1016/j.scico.2023.103033.

[4] Cheng, Q., et al., 2019. An accuracy degradation analysis of ball screw mechanism considering the time-varying motion and loading working conditions. Mechanism and Machine Theory ScienceDirect.

[5] 03.080.10 Maintenance services. Facilities management, .M.o.h.r., 2025. En 15628:2024, maintenance - qualification of maintenance personnel.

[6] Feng, H., et al., 2022. Integration of the mape-k loop in digital twins. PDF available online.

[7] Gurocak, H., 2015. Industrial motion control: motor selection, drives, controller tuning, applications. John Wiley & Sons.

[8] Haber, A., Ringert, J.O., Rumpe, B., 2012. MontiArc - Architectural Modeling of Interactive Distributed and Cyber-Physical Systems. Technical Report AIB-2012-03. RWTH Aachen University. URL: http://www.se-rwth.de/publications/MontiArc-Architectural-Modeling-of-Interactive-Distributed-and-Cyber-Physical-S pdf.

[9] He, D., Pang, Y., Lodewijks, G., 2016. Speed control of belt conveyors during transient operation. Powder Technology doi:10.1016/j.powtec.2016.07.004.

[10] Mallioris, P., et al., 2024. Predictive maintenance in industry 4.0: A systematic multi-... Journal of Manufacturing Systems ScienceDirect.

[11] Pfeiffer, J., Zhang, J., Combemale, B., Michael, J., Rumpe, B., Wimmer, M., Wortmann, A., 2025. Towards a Unifying Reference Model for Digital Twins of Cyber-Physical Systems, in: 2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA), pp. 1–4. URL: http://www.se-rwth.de/publications/Towards-a-Unifying-Reference-Model-for-Digital-Twins-of-Cyber-Physical-Systems pdf, doi:10.1109/ETFA65518.2025.11205646.

[12] Theissler, A., et al., 2021. Predictive maintenance enabled by machine learning. Reliability Engineering & System Safety PII: S0951832021003835.

[13] Zhao, J., et al., 2018. Research on the precision loss of ball screw with short-time heavy load operation. Advances in Mechanical Engineering doi:10.1177/1687814018817902.

Appendix

7.1. Components

7.1.1. Encoder Motor

- Inputs and outputs as given by the Fischertechnik specification:
  - In:
    - `forward`: Boolean
    - `reverse`: Boolean

  - Out:
    * `encoder1Level`: Boolean (high/low signal)
    * `encoder2Level`: Boolean

- Preprocessed:
  - Relative steps measured by the encoder, derived from `encoder1Level` & `encoder2Level`.

7.1.2. Stations list

Vacuum Gripper.

Multiprocessing Station.

3D-Gripper.

Conveyor Belt.

High Bay.

Punching Machine.

Sorting Line.

Indexed Line.

Machine port desc	Machine port number	Direction	Type	MQTT topic
reference switch vertical axis	I1	Out	Boolean	`/${machine-id}/ref-switch-vertical`
reference switch horizontal axis	I2	Out	Boolean	`/${machine-id}/ref-switch-horizontal`
reference switch rotate	I3	Out	Boolean	`/${machine-id}/ref-switch-rotation`
encoder vertical axis impulse 1	B1	Out	Boolean	`/${machine-id}/motor-vertical-enc-1`
encoder vertical axis impulse 2	B2	Out	Boolean	`/${machine-id}/motor-vertical-enc-2`
encoder horizontal axis impulse 1	B3	Out	Boolean	`/${machine-id}/motor-horizontal-enc-1`
encoder horizontal axis impulse 2	B4	Out	Boolean	`/${machine-id}/motor-horizontal-enc-2`
encoder rotate impulse 1	B5	Out	Boolean	`/${machine-id}/motor-rotation-enc-1`
encoder rotate impulse 2	B6	Out	Boolean	`/${machine-id}/motor-rotation-enc-2`
Vertical position	Virtual	Out	Integer	`/${machine-id}/motor-vertical-pos`
Horizontal position	Virtual	Out	Integer	`/${machine-id}/motor-horizontal-pos`
Rotation position	Virtual	Out	Integer	`/${machine-id}/motor-rotation-pos`
motor vertical axis up	Q1	In	Boolean	`/${machine-id}/move-vertical-up`
motor vertical axis down	Q2	In	Boolean	`/${machine-id}/move-vertical-down`
motor horizontal axis backward	Q3	In	Boolean	`/${machine-id}/move-horizontal-up`
motor horizontal axis forward	Q4	In	Boolean	`/${machine-id}/move-horizontal-down`
motor rotate clockwise	Q5	In	Boolean	`/${machine-id}/move-rotation-clockwise`
motor rotate counterclockwise	Q6	In	Boolean	`/${machine-id}/move-rotation-counterclockwise`
compressor	Q7	In	Boolean	`/${machine-id}/enable-compressor`
Valve vacuum	Q8	In	Boolean	`/${machine-id}/enable-valve`

Table 1: Vacuum Gripper

Machine port desc	Machine port number	Direction	Type	MQTT topic
reference switch turn-table (position vacuum)	I1	Out	Integer	`/${machine-id}/ref-switch-rotation-at-vacuum`
reference switch turn-table (position belt)	I2	Out	Integer	`/${machine-id}/ref-switch-rotation-at-belt`
light-barrier end of conveyor belt	I3	Out	Boolean	`/${machine-id}/light-barrier-conveyor-end`
reference switch turn-table (position saw)	I4	Out	Integer	`/${machine-id}/ref-switch-turn-table-at-saw`
reference switch vacuum (position turn-table)	I5	Out	Boolean	`/${machine-id}/ref-switch-vacuum-at-turn-table`
reference switch oven feeder inside	I6	Out	Boolean	`/${machine-id}/ref-switch-feeder-inside`
reference switch oven feeder outside	I7	Out	Boolean	`/${machine-id}/ref-switch-feeder-outside`
reference switch vacuum (position oven)	I8	Out	Integer	`/${machine-id}/ref-switch-vacuum-at-oven`
light-barrier oven	I9	Out	Boolean	`/${machine-id}/light-barrier-oven`
motor turn-table clockwise	Q1 (M1)	In	Boolean	`/${machine-id}/move-turn-table-clockwise`
motor turn-table counterclock-wise	Q2 (M1)	In	Boolean	`/${machine-id}/move-turn-table-counterclockwise`
motor conveyor belt forward	Q3 (M2)	In	Boolean	`/${machine-id}/move-converyor-forward`
motor saw	Q4 (M3)	In	Boolean	`/${machine-id}/enable-saw`
motor oven feeder retract	Q5 (M4)	In	Boolean	`/${machine-id}/move-oven-feeder-retract`
motor oven feeder extend	Q6 (M4)	In	Boolean	`/${machine-id}/move-oven-feeder-extend`
motor vacuum towards oven	Q7 (M5)	In	Boolean	`/${machine-id}/move-vacuum-to-oven`
motor vacuum towards turn-table	Q8 (M5)	In	Boolean	`/${machine-id}/move-vacuum-to-turn-table`
light oven	Q9	In	Boolean	`/${machine-id}/enable-oven-light`
compressor	Q10	In	Boolean	`/${machine-id}/enable-compressor`
valve vacuum	Q11	In	Boolean	`/${machine-id}/enable-valve-vacuum`
valve lowering	Q12	In	Boolean	`/${machine-id}/enable-valve-move-vacuum`
valve oven door	Q13	In	Boolean	`/${machine-id}/enable-valve-move-oven-door`
valve feeder	Q14	In	Boolean	`/${machine-id}/enable-valve-move-feeder`

Table 2: Multiprocessing Station

Machine port desc	Machine port number	Direction	Type	MQTT topic
reference switch claw	I1	Out	Boolean	`/${machine-id}/ref-switch-claw`
pulse counter gripper (TODO: weird name?)	I2	Out	Integer	`/${machine-id}/enc-counter-claw`
reference switch grip arm	I3	Out	Boolean	`/${machine-id}/ref-switch-gripper`
pulse counter grip arm (TODO: weird name?)	I4	Out	Integer	`/${machine-id}/enc-counter-gripper`
reference switch vertical axis	I5	Out	Boolean	`/${machine-id}/ref-switch-vertical`
reference switch turntable	I6	Out	Boolean	`/${machine-id}/ref-switch-rotation`
encoder vertical axis impulse 1	B1	Out	Boolean	`/${machine-id}/motor-vertical-enc-1`
encoder vertical axis impulse 2	B2	Out	Boolean	`/${machine-id}/motor-vertical-enc-2`
encoder turntable impulse 1	B3	Out	Boolean	`/${machine-id}/motor-rotation-enc-1`
encoder turntable impulse 2	B4	Out	Boolean	`/${machine-id}/motor-rotation-enc-2`
position vertical	Virtual	Out	Integer	`/${machine-id}/motor-vertical-pos`
position rotation	Virtual	Out	Integer	`/${machine-id}/motor-rotation-pos`
motor gripper open	Q1 (M1)	In	Boolean	`/${machine-id}/open-gripper`
motor gripper close	Q2 (M1)	In	Boolean	`/${machine-id}/close-gripper`
motor grip arm forward	Q3 (M2)	In	Boolean	`/${machine-id}/move-horizontal-forward`
motor grip arm back	Q4 (M2)	In	Boolean	`/${machine-id}/move-horizontal-backward`
motor vertical axis down	Q5 (M3)	In	Boolean	`/${machine-id}/move-vertical-up`
motor vertical axis up	Q6 (M3)	In	Boolean	`/${machine-id}/move-vertical-down`
motor turntable right	Q7 (M4)	In	Boolean	`/${machine-id}/move-rotation-clockwise`
motor turntable left	Q8 (M4)	In	Boolean	`/${machine-id}/move-rotation-counterclockwise`

Table 3: 3D-Gripper

Machine port desc	Machine port number	Direction	Type	MQTT topic
phototransistor feed station	I1	Out	Boolean	`/${machine-id}/phototransistor-feed-station`
phototransistor swap station	I2	Out	Boolean	`/${machine-id}/phototransistor-swap-station`
pulse button	I3	Out	Boolean	`/${machine-id}/pulse-button`
motor conveyor belt forward	Q1	In	Boolean	`/${machine-id}/move-conveyor-forward`
motor conveyor belt backward	Q2	In	Boolean	`/${machine-id}/move-conveyor-backward`

Table 4: Conveyor Belt

Machine port desc	Machine port number	Direction	Type	MQTT topic
reference switch horizontal axis	I1	Out	Boolean	`/${machine-id}/ref-switch-horizontal`
light-barrier inside	I2	Out	Boolean	`/${machine-id}/light-barrier-inside`
light-barrier outside	I3	Out	Boolean	`/${machine-id}/light-barrier-outside`
reference switch vertical axis	I4	Out	Boolean	`/${machine-id}/ref-switch-vertical`
trail sensor (signal 1, lower)	A1	Out	Boolean	`/${machine-id}/trail-sensor-lower`
trail sensor (signal 2, upper)	A2	Out	Boolean	`/${machine-id}/trail-sensor-upper`
Encoder horizontal impulse 1	B1	Out	Boolean	`/${machine-id}/horizontal-enc-1`
Encoder horizontal impulse 2	B2	Out	Boolean	`/${machine-id}/horizontal-enc-2`
Encoder vertical impulse 1	B3	Out	Boolean	`/${machine-id}/vertical-enc-1`
Encoder vertical impulse 2	B4	Out	Boolean	`/${machine-id}/vertical-enc-2`
reference switch cantilever front	I5	Out	Boolean	`/${machine-id}/ref-switch-cantilever-front`
reference switch cantilever back	I6	Out	Boolean	`/${machine-id}/ref-switch-cantilever-back`
position vertical	Virtual	Out	Integer	`/${machine-id}/vertical-pos`
position horizontal	Virtual	Out	Integer	`/${machine-id}/horizontal-pos`
motor conveyor belt forward	Q1	In	Boolean	`/${machine-id}/move-conveyor-forward`
motor conveyor belt backward	Q2	In	Boolean	`/${machine-id}/move-conveyor-backward`
motor horizontal towards rack	Q3	In	Boolean	`/${machine-id}/move-arm-to-rack`
motor horizontal towards conveyor belt	Q4	In	Boolean	`/${machine-id}/move-arm-to-conveyor`
motor vertical axis downward	Q5	In	Boolean	`/${machine-id}/move-vertical-down`
motor vertical axis upward	Q6	In	Boolean	`/${machine-id}/move-vertical-up`
motor cantilever forward	Q7	In	Boolean	`/${machine-id}/move-cantilever-forward`
motor cantilever backward	Q8	In	Boolean	`/${machine-id}/move-cantilever-backward`

Table 5: High Bay

Machine port desc	Machine port number	Direction	Type	MQTT topic
phototransistor goods in/out	I1	Out	Boolean	`/${machine-id}/phototransistor-goods-in-out`
phototransistor punching machine	I2	Out	Boolean	`/${machine-id}/phototransistor-punching-machine`
switch punching machine up	I3	Out	Boolean	`/${machine-id}/switch-punching-machine-up`
switch punching machine down	I4	Out	Boolean	`/${machine-id}/switch-punching-machine-down`
motor conveyor belt forward	Q1	In	Boolean	`/${machine-id}/move-conveyor-forward`
motor conveyor belt backward	Q2	In	Boolean	`/${machine-id}/move-conveyor-backward`
motor punching machine up	Q3	In	Boolean	`/${machine-id}/move-punching-machine-up`
motor punching machine down	Q4	In	Boolean	`/${machine-id}/move-punching-machine-down`

Table 6: Punching Machine

Machine port desc	Machine port number	Direction	Type	MQTT topic
pulse counter	I1	Out	Integer	`/${machine-id}/pulse-counter`
light-barrier inlet	I2	Out	Boolean	`/${machine-id}/light-barrier-inlet`
light-barrier behind color sensor	I3	Out	Boolean	`/${machine-id}/light-barrier-behind-color-sensor`
color sensor	A4	Out	Integer	`/${machine-id}/color-sensor`
light-barrier white	I5	Out	Boolean	`/${machine-id}/light-barrier-white`
light-barrier red	I6	Out	Boolean	`/${machine-id}/light-barrier-red`
light-barrier blue	I7	Out	Boolean	`/${machine-id}/light-barrier-blue`
motor conveyor belt	Q1	In	Boolean	`/${machine-id}/move-conveyor`
compressor	Q2	In	Boolean	`/${machine-id}/enable-compressor`
valve first ejector	Q3	In	Boolean	`/${machine-id}/enable-valve-first-ejector`
valve second ejector	Q4	In	Boolean	`/${machine-id}/enable-valve-second-ejector`
valve third ejector	Q5	In	Boolean	`/${machine-id}/enable-valve-third-ejector`

Table 7: Sorting Line

Machine port desc	Machine port number	Direction	Type	MQTT topic
push-button slider 1 front	I1	Out	Boolean	`/${machine-id}/button-slider-1-front`
push-button slider 1 rear	I2	Out	Boolean	`/${machine-id}/button-slider-1-rear`
push-button slider 2 front	I3	Out	Boolean	`/${machine-id}/button-slider-2-front`
push-button slider 2 rear	I4	Out	Boolean	`/${machine-id}/button-slider-2-rear`
phototransistor slider 1	I5	Out	Boolean	`/${machine-id}/phototransistor-slider-1`
phototransistor milling machine	I6	Out	Boolean	`/${machine-id}/phototransistor-milling-machine`
phototransistor loading station	I7	Out	Boolean	`/${machine-id}/phototransistor-loading-station`
phototransistor drilling machine	I8	Out	Boolean	`/${machine-id}/phototransistor-drilling-machine`
phototransistor conveyor belt swap	I9	Out	Boolean	`/${machine-id}/phototransistor-conveyor-swap`
motor slider 1 backward	Q1	In	Boolean	`/${machine-id}/move-slider-1-backward`
motor slider 1 forward	Q2	In	Boolean	`/${machine-id}/move-slider-1-forward`
motor slider 2 backward	Q3	In	Boolean	`/${machine-id}/move-slider-2-backward`
motor slider 2 forward	Q4	In	Boolean	`/${machine-id}/move-slider-2-forward`
motor conveyor belt feed	Q5	In	Boolean	`/${machine-id}/move-conveyor-feed`
motor conveyor belt milling machine	Q6	In	Boolean	`/${machine-id}/move-conveyor-milling-machine`
motor milling machine	Q7	In	Boolean	`/${machine-id}/move-milling-machine`
motor conveyor belt drilling machine	Q8	In	Boolean	`/${machine-id}/move-conveyor-drilling-machine`
motor drilling machine	Q9	In	Boolean	`/${machine-id}/move-drilling-machine`
motor conveyor belt swap	Q10	In	Boolean	`/${machine-id}/move-conveyor-swap`

Table 8: Indexed Line